



Sistema de Gestión de Colección de Láminas

API REST con Spring Boot y MySQL

Examen Transversal

Asignatura: Desarrollo de Software Web II

Profesor: Boris Marcelo Belmar Muñoz

Integrantes: Valeria Gómez y Otton Lucena

Grupo 2

Septiembre de 2026

Índice

1. Introducción.....	3
1.1 Justificación de la tecnología.....	3
2. Desarrollo.....	3
2.1 Configuración del proyecto y conexión a la base de datos.....	3
2.2 Arquitectura y modelo de datos.....	4
2.3 API REST y funcionalidades especiales.....	5
2.4 Validaciones y auditoría.....	6
3. Pruebas.....	8
4. Conclusión.....	10
5. Bibliografía.....	10
Anexo A. Capturas de las pruebas.....	11
A.1 Peticiones de la colección de Postman.....	11
A.2 Proyecto, base de datos y suites automáticas.....	13

1. Introducción

Quienes coleccionan láminas necesitan saber cuáles les faltan y cuáles tienen repetidas para decidir qué comprar e intercambiar, algo difícil de controlar a mano cuando el álbum tiene cientos de láminas. Nuestro objetivo fue construir una API REST en Spring Boot y MySQL que administre álbumes y láminas, permita cargarlas en lote y registrar las obtenidas, y entregue las faltantes, las repetidas con su cantidad de repetidas y un resumen del avance, validando los datos y registrando cada cambio. El alcance es el que fija el enunciado, un backend REST sin frontend obligatorio; la autenticación de usuarios quedó fuera.

1.1 Justificación de la tecnología

Usamos Spring Boot 4.1.1 y Java 21, versión de soporte extendido, porque el enunciado pide un proyecto Maven creado con Spring Initializr (Spring, s. f.-c) y porque Spring Web MVC, Spring Data JPA y Bean Validation cubren los requisitos sin infraestructura propia (Spring, s. f.-a). MySQL 8.4, también de soporte extendido, es obligatoria según el enunciado y el profesor, y corresponde a un problema relacional: cada lámina pertenece a un álbum y su número no se repite en él, reglas que MySQL garantiza con claves foráneas, UNIQUE y CHECK (Oracle Corporation, s. f.).

Para la auditoría seguimos la indicación del profesor de usar migraciones: Flyway 12.4 aplica scripts SQL versionados y registra cada cambio del esquema en flyway_schema_history (Redgate Software, s. f.), y JPA Auditing con Hibernate Envers 7.4 registran quién y cuándo cambió cada registro, incluso eliminado (Hibernate, s. f.). Docker Compose levanta todo con un comando (Docker Inc., s. f.); springdoc-openapi 3.1, la línea compatible con Spring Boot 4, documenta la API (springdoc-openapi, s. f.); Postman guarda las pruebas en una colección JSON versionada, que también corre en la terminal con Newman (Postman, s. f.), y Testcontainers prueba contra MySQL 8.4 real en lugar de una base en memoria (Testcontainers, s. f.).

2. Desarrollo

2.1 Configuración del proyecto y conexión a la base de datos

Generamos el proyecto en Spring Initializr con Maven, Java 21, Spring Boot 4.1.1, grupo cl.ipss, artefacto coleccion-laminas y las dependencias Spring Web, Spring Data JPA, MySQL Driver, Validation, Flyway Migration y Lombok (Figura 1). Al pom.xml agregamos a mano solo hibernate-envers, con la versión que administra Spring Boot, y springdoc-openapi 3.1.0; flyway-mysql habilita a Flyway para MySQL y las dependencias de prueba tienen alcance test (Figura 2).

El archivo application.properties toma la conexión de las variables DB_URL, DB_USER y DB_PASSWORD, con valores por defecto de desarrollo; así el mismo archivo sirve en el equipo local y en Docker, y las credenciales quedan en un .env que no se versiona (Figura 3). Con ddl-auto=validate, Hibernate no toca el esquema y solo verifica al

arrancar que coincide con las entidades; con open-in-view=false ninguna consulta ocurre fuera de los servicios.

El archivo compose.yaml define el servicio mysql (imagen mysql:8.4, volumen mysql-data y healthcheck) y el servicio api, construido con un Dockerfile de dos etapas (JDK 21 para compilar y JRE 21 con un usuario sin privilegios para ejecutar), que arranca solo cuando MySQL está sano. En la verificación final del 13 de septiembre de 2026, hecha desde cero con volúmenes nuevos y docker compose up -d --build, MySQL quedó healthy y la API activa (Figura 4); Flyway aplicó 5 migraciones y la aplicación arrancó en 15,729 segundos (Figura 45).

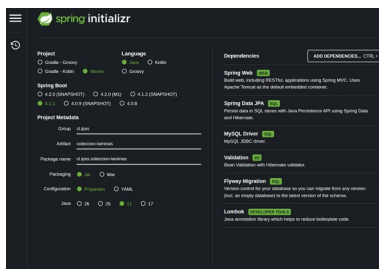


Figura 1: Proyecto en Spring Initializr (ev. 01).

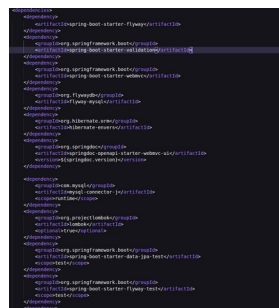


Figura 2: Dependencias del pom.xml (ev. 02).

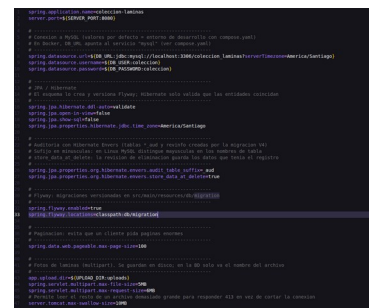


Figura 3: Configuración en application.properties (ev. 03).

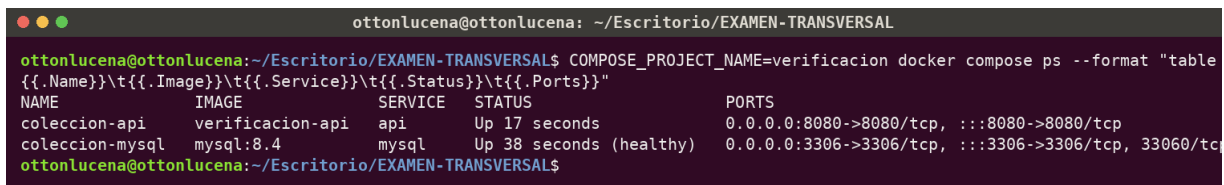


Figura 4: Contenedores de MySQL y de la API en ejecución (ev. 05).

2.2 Arquitectura y modelo de datos

La aplicación se organiza en las capas controller, service y repository (Figura 40). Los cinco controladores delegan en un servicio, sin lógica de negocio ni acceso a repositorios, y los tres que reciben un cuerpo JSON (álbumes, láminas y colección) lo validan con @Valid. Cada servicio tiene interfaz e implementación, concentra las reglas de negocio y define las transacciones (@Transactional en las escrituras y readOnly en las lecturas); los repositorios extienden JpaRepository (Spring, s. f.-b). Entre capas viajan DTOs definidos como record y convertidos por mappers manuales, así ninguna entidad JPA se expone en la API.

El enunciado pide una API REST y no exige una interfaz gráfica, por lo que no desarrollamos un frontend. La interfaz de usuario del sistema es Swagger UI (/swagger-ui.html), que muestra y permite ejecutar las 21 operaciones con sus parámetros, ejemplos y códigos de respuesta, junto con la colección de Postman de 30 peticiones. Para quien la usa, todos los errores llegan en español y con un formato único.

Album guarda nombre, que es único, imagen, fecha de lanzamiento, tipo de láminas, total, editorial y descripción; Lamina guarda número, nombre, tipo, cantidad de copias y el archivo de su foto opcional (Figura 5). La relación muchos a uno, con carga diferida, es la clave foránea album_id con ON DELETE CASCADE, y UNIQUE (album_id, numero) impide repetir un número en el álbum. Ambas entidades heredan de EntidadAuditable (creado_en, creado_por, modificado_en y modificado_por) y el historial se guarda en revinfo, album_aud y lamina_aud. El estado de una lámina sale de un solo campo, cantidad: 0 es faltante, 1 o más es obtenida y más de 1 es repetida, con cantidad - 1 repetidas. Un contador, en vez de tablas separadas, evita estados inconsistentes, y CHECK (cantidad >= 0) lo protege en la base.

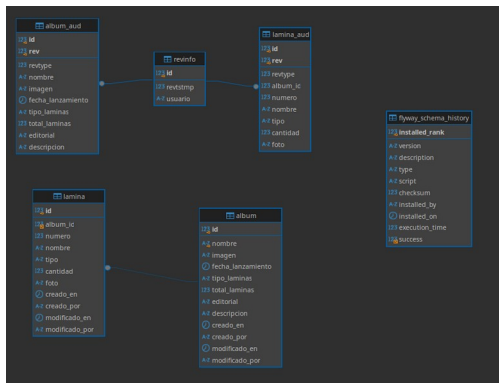


Figura 5: Diagrama entidad-relación del esquema creado por Flyway (ev. 10).

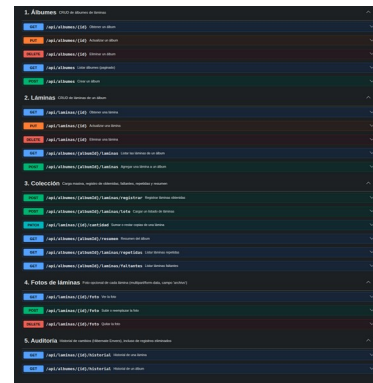


Figura 6: Swagger UI con las operaciones agrupadas (ev. 11).

2.3 API REST y funcionalidades especiales

La API expone 21 operaciones sobre 13 rutas, todas bajo el prefijo /api (Tabla 1). Crear un álbum o una lámina responde 201 con el encabezado Location y la carga en lote, 201 con las láminas creadas; las lecturas y actualizaciones responden 200 y las eliminaciones 204. El listado de álbumes es paginado, con un máximo de 100 elementos por página, y Swagger UI agrupa las operaciones en 5 secciones (Figura 6).

Tabla 1: Operaciones de la API, sin el prefijo /api (fuente: /v3/api-docs, evidencia 11)

Método	Ruta	Función
POST	/albumes	Crear álbum
GET	/albumes	Listar álbumes paginados
GET	/albumes/{id}	Obtener un álbum
PUT	/albumes/{id}	Actualizar un álbum
DELETE	/albumes/{id}	Eliminar álbum, láminas y fotos
GET	/albumes/{albumId}/resumen	Resumen y porcentaje de avance
GET	/albumes/{id}/historial	Historial del álbum (Envers)
POST	/albumes/{albumId}/laminas	Agregar una lámina
GET	/albumes/{albumId}/laminas	Listar láminas (filtro ?tipo=)
POST	/albumes/{albumId}/laminas/lote	Carga en lote, todo o nada
POST	/albumes/{albumId}/laminas/registrar	Registrar obtenidas por número

Método	Ruta	Función
GET	/albumes/{albumId}/laminas/faltantes	Láminas faltantes
GET	/albumes/{albumId}/laminas/repetidas	Repetidas y cantidad de repetidas
GET	/laminas/{id}	Obtener una lámina
PUT	/laminas/{id}	Actualizar una lámina
DELETE	/laminas/{id}	Eliminar una lámina
PATCH	/laminas/{id}/cantidad	Sumar o restar copias
POST	/laminas/{id}/foto	Subir foto (multipart)
GET	/laminas/{id}/foto	Ver la foto
DELETE	/laminas/{id}/foto	Quitar la foto
GET	/laminas/{id}/historial	Historial de la lámina (Envers)

La carga en lote es todo o nada: @NumerosUnicos, nuestro validador personalizado, rechaza con 400 los números repetidos en la lista; luego el servicio consulta una vez los números ya catalogados y, si alguna posición excede el total o ya existe, responde 409 con los errores por posición (laminas[0].numero, laminas[1].numero) sin guardar nada; si no hay conflictos, guarda todo con saveAll dentro de una transacción. El registro de obtenidas recibe números como {"numeros": [1, 5, 5, 12]}, busca las láminas con una consulta IN y suma una copia por aparición, o rechaza todo con 409 si hay números no catalogados; el PATCH de cantidad aplica un delta sin bajar de cero.

Faltantes y repetidas se resuelven en la base de datos, no en memoria: dos métodos derivados de Spring Data en LaminaRepository generan consultas con WHERE cantidad = 0 y WHERE cantidad > 1, y cada repetida informa su cantidad de repetidas. Las faltantes incluyen además los números aún no catalogados, calculados a partir de una consulta que trae solo la columna numero, y el resumen sale de una única consulta JPQL con count y sum(case ...). Filtrar en Java obligaría a cargar todas las láminas del álbum en cada consulta, mientras que MySQL usa el índice de UNIQUE (album_id, numero) y devuelve solo las filas pedidas.

La foto se sube como multipart/form-data, en el campo archivo, y se guarda en el volumen uploads con un nombre UUID; la base guarda solo ese nombre. Descartamos Base64 y BLOB porque Base64 aumenta el tamaño en un tercio y engrosa filas y respaldos, y el profesor lo indicó como mala práctica. El tipo se valida por los primeros bytes (JPEG FF D8 FF, PNG 89 50 4E 47, WEBP RIFF...WEBP) y no por la extensión, el máximo es 5 MB, la ruta se normaliza contra path traversal y los archivos se borran solo tras el commit, o tras el rollback si la subida falla.

2.4 Validaciones y auditoría

Validamos en tres niveles: Bean Validation en los DTOs (@NotBlank, @NotNull, @NotEmpty, @Size, @Min, @Max, @PastOrPresent y @Pattern), activada con @Valid (Bean Validation, s. f.), junto con el validador personalizado @NumerosUnicos (@Constraint y ConstraintValidator), que responden 400; reglas de negocio en los servicios, que responden 409 (nombre de álbum único, número no mayor que el total y

La auditoría tiene tres capas. La principal, pedida por el profesor, son las migraciones: el esquema se crea solo con Flyway (V1 álbum, V2 lámina, V3 columnas de auditoría, V4 tablas de Envers y V5 datos de ejemplo), queda versionado en Git y cada cambio se registra en `flyway_schema_history` con versión, checksum, fecha, duración y resultado; las 5 migraciones figuran con `success = 1` (Figura 7). Como Flyway verifica el checksum de lo ya aplicado, alterar una migración impide arrancar la aplicación, así que el historial del esquema no se puede reescribir.

```

ottonlucena@ottonlucena: ~/Escritorio/EXAMEN-TRANSVERSAL

ottonlucena@ottonlucena:~/Escritorio/EXAMEN-TRANSVERSAL$ COMPOSE_PROJECT_NAME=verificacion docker compose exec -T -e MYSQL_PWD=coleccion mysql
mysql -ucoleccion coleccion_laminas -t -e "SELECT installed_rank, version, description, script, installed_on, execution_time, success FROM fly
ay_schema_history;"
+-----+-----+-----+-----+-----+-----+-----+
| installed_rank | version | description                | script                | installed_on          | execution_time | success |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | 1 | crear tabla album          | V1_crear_tabla_album.sql | 2026-09-13 20:40:17 | 66 | 1 |
| 2 | 2 | crear tabla lamina         | V2_crear_tabla_lamina.sql | 2026-09-13 20:40:17 | 111 | 1 |
| 3 | 3 | agregar columnas auditoria | V3_agregar_columnas_auditoria.sql | 2026-09-13 20:40:17 | 200 | 1 |
| 4 | 4 | crear tablas envers       | V4_crear_tablas_envers.sql | 2026-09-13 20:40:18 | 169 | 1 |
| 5 | 5 | datos semilla             | V5_datos_semilla.sql      | 2026-09-13 20:40:18 | 42 | 1 |
+-----+-----+-----+-----+-----+-----+-----+

ottonlucena@ottonlucena:~/Escritorio/EXAMEN-TRANSVERSAL$

```

[illegible]

The screenshot shows the AWS IAM console 'Groups' page. The 'AWS managed' group is selected, and its permissions are displayed. The 'Permissions summary' shows 1 policy attached: 'AmazonS3OutpostsFullAccess'. The 'Policies' section lists the 'AmazonS3OutpostsFullAccess' policy, which is an AWS managed policy with 1,000 permissions. The 'Permissions' section shows the policy is attached to the 'AWS managed' group.

[illegible]

7

3. Pruebas

Probamos en tres niveles el 13 de septiembre de 2026, sobre un entorno recién creado: ./mvnw verify ejecutó 11 pruebas con 0 fallos (7 unitarias con Mockito, 3 de la capa web con @WebMvcTest y 1 de integración con Testcontainers, que aplica las migraciones en MySQL 8.4 y valida el esquema); pruebas/e2e.sh hizo 110 comprobaciones con 0 fallos, incluidas consultas a flyway_schema_history y lamina_aud, y Newman, el ejecutor de línea de comandos de Postman, aprobó 30 de 30 peticiones y 31 de 31 aserciones (Figura 20; las salidas completas están en el Anexo A).

La Tabla 2 muestra, para cada petición de la colección de Postman, el código esperado, el obtenido con Newman y la figura con su captura: las más representativas están a continuación y el resto en el Anexo A. El caso exitoso de GET /api/albumes/{id} y GET /api/laminas/{id} no tiene una petición propia en la colección; lo ejecuta el script end-to-end con respuesta 2xx (Figura 46).

Tabla 2: Peticiones de la colección de Postman: código esperado y obtenido con Newman (fuente: evidencia 46)

N°	Petición	Caso	Esp	Obt.	Evid.
12	POST /albumes	Crear álbum	201	201	Fig. 21
13	POST /albumes	Validación de datos	400	400	Fig. 11
14	POST /albumes	Nombre duplicado	409	409	Fig. 22
15	GET /albumes? page=0&size=5&sort=nombre	Listar álbumes paginado	200	200	Fig. 23
16	GET /albumes/{id}	Álbum inexistente	404	404	Fig. 24
17	PUT /albumes/{id}	Actualizar álbum	200	200	Fig. 25
18	POST /albumes/{albumId}/laminas	Crear lámina	201	201	Fig. 26
19	POST /albumes/{albumId}/laminas	Número fuera de rango	409	409	Fig. 27
20	GET /albumes/{albumId}/laminas? tipo=NORMAL	Listar láminas por tipo	200	200	Fig. 28
21	PUT /laminas/{id}	Actualizar lámina	200	200	Fig. 29
22	DELETE /laminas/{id}	Eliminar lámina	204	204	Fig. 30
23	POST /albumes/{albumId}/laminas/lote	Carga en lote	201	201	Fig. 12
24	POST /albumes/{albumId}/laminas/lote	Lote con conflictos	409	409	Fig. 13
25	POST /albumes/{albumId}/laminas/lote	Lote con números repetidos	400	400	Fig. 31
26	POST /albumes/{albumId}/laminas/regar	Registrar obtenidas	200	200	Fig. 14
27	POST /albumes/{albumId}/laminas/regar	Registrar número no catalogado	409	409	Fig. 32
28	PATCH /laminas/{id}/cantidad	Restar una copia	200	200	Fig. 33
29	PATCH /laminas/{id}/cantidad	Cantidad negativa	409	409	Fig. 34
30	GET /albumes/{albumId}/laminas/faltantes	Láminas faltantes	200	200	Fig. 15
31	GET /albumes/{albumId}/laminas/repetidas	Láminas repetidas	200	200	Fig. 16
32	GET /albumes/{albumId}/resumen	Resumen del álbum	200	200	Fig. 17
33	POST /laminas/{id}/foto	Subir foto	200	200	Fig. 18

N°	Petición	Caso	Esp	Obt.	Evid.
34	GET /laminas/{id}/foto	Ver foto	200	200	Fig. 35
35	POST /laminas/{id}/foto	Archivo que no es imagen	415	415	Fig. 19
36	POST /laminas/{id}/foto	Archivo mayor a 5 MB	413	413	Fig. 36
37	DELETE /laminas/{id}/foto	Quitar foto	204	204	Fig. 37
38	PUT /albumes/{id}	Auditoría con X-Usuario	200	200	Fig. 8
39	GET /laminas/{id}/historial	Historial de lámina eliminada	200	200	Fig. 38
40	DELETE /albumes/{id}	Eliminar álbum	204	204	Fig. 39
41	GET /albumes/{id}/historial	Historial de álbum eliminado	200	200	Fig. 9

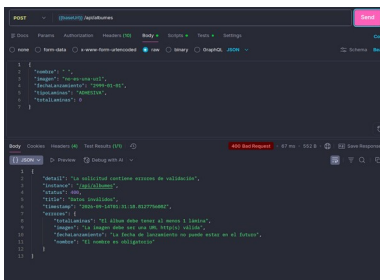


Figura 11: Validación de datos (400) (ev. 13).

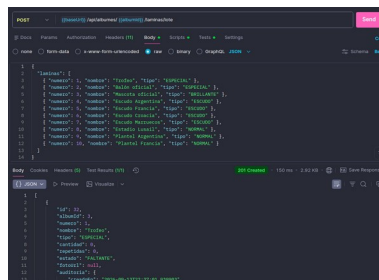


Figura 12: Carga en lote (201) (ev. 23).

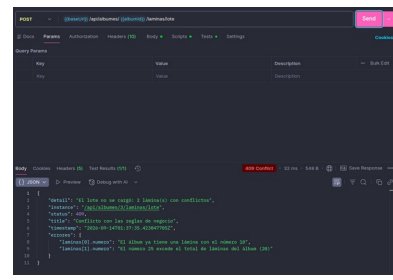


Figura 13: Lote con conflictos (409) (ev. 24).

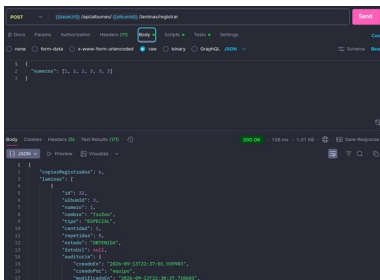


Figura 14: Registrar obtenidas (200) (ev. 26).

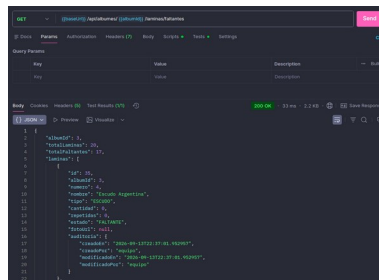


Figura 15: Láminas faltantes (200) (ev. 30).

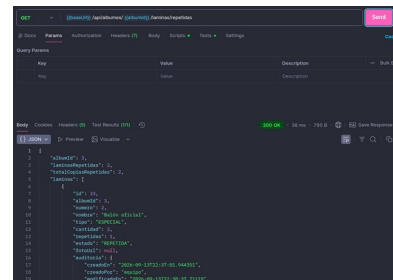


Figura 16: Láminas repetidas (200) (ev. 31).

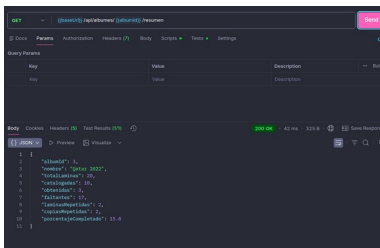


Figura 17: Resumen del álbum (200) (ev. 32).

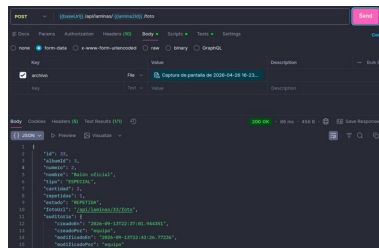


Figura 18: Subir foto (200) (ev. 33).

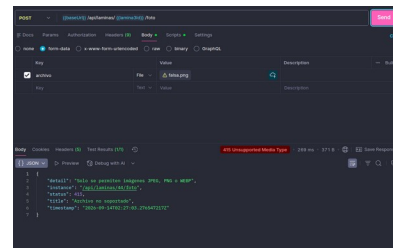


Figura 19: Archivo que no es imagen (415) (ev. 35).

```

ottonlucena@ottonlucena: ~/Escritorio/EXAMEN-TRANSVERSAL

ottonlucena@ottonlucena:~/Escritorio/EXAMEN-TRANSVERSAL$ for f in docs/evidencias/45-mvnw-verify.txt
docs/evidencias/44-script-e2e.txt docs/evidencias/46-newman.txt; do grep -H -E "\[INFO\]" (Tests ru
n: [0-9]+, Failures: [0-9]+, Errors: [0-9]+, Skipped: [0-9]+$|BUILD)|RESULTADO|)+(requests|assertio
ns) |" "$f"; done
docs/evidencias/45-mvnw-verify.txt:[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
docs/evidencias/45-mvnw-verify.txt:[INFO] BUILD SUCCESS
docs/evidencias/44-script-e2e.txt:===== RESULTADO: 110 OK, 0 FAIL =====
docs/evidencias/46-newman.txt:|           requests |           30 |           0 |
docs/evidencias/46-newman.txt:|           assertions |           31 |           0 |
ottonlucena@ottonlucena:~/Escritorio/EXAMEN-TRANSVERSAL$

```

Figura 20: Totales de ./mvnw verify, del script e2e y de Newman (Postman) (ev. 47).

4. Conclusión

Cumplimos los requerimientos del enunciado: la API se conecta a MySQL 8.4 en Docker, ofrece el CRUD con foto opcional, la carga en lote y los listados de faltantes y repetidas calculados en la base, separa la lógica de negocio del acceso a datos, valida en tres niveles y audita con migraciones, JPA Auditing y Envers; tres suites de pruebas lo verifican.

Resolvimos tres problemas: los enum que Hibernate 7 mapeaba al tipo ENUM de MySQL, corregidos con `@JdbcTypeCode(SqlTypes.VARCHAR)`; las láminas borradas por la cascada de la base, que Envers no auditaba y ahora se eliminan como entidades, y las fechas de auditoría, truncadas a microsegundos por `DATETIME(6)`. Quedan como mejoras la autenticación, pues X-Usuario identifica pero no autentica, y un frontend web.

5. Bibliografía

- Bean Validation. (s. f.). *Jakarta Bean Validation 3.1*. <https://beanvalidation.org/3.1/>
- Docker Inc. (s. f.). *Docker Compose*. <https://docs.docker.com/compose/>
- Hibernate. (s. f.). *Hibernate ORM documentation* (Versión 7.4). <https://hibernate.org/orm/documentation/>
- Nottingham, M., Wilde, E. y Dalal, S. (2023). *Problem details for HTTP APIs* (RFC 9457). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9457>
- Oracle Corporation. (s. f.). *MySQL 8.4 reference manual*. <https://dev.mysql.com/doc/refman/8.4/en/>
- Postman. (s. f.). *Postman Learning Center*. <https://learning.postman.com/>
- Redgate Software. (s. f.). *Flyway documentation*. <https://documentation.red-gate.com/flyway>
- Spring. (s. f.-a). *Spring Boot reference documentation* (Versión 4.1). <https://docs.spring.io/spring-boot/>
- Spring. (s. f.-b). *Spring Data JPA reference documentation*. <https://docs.spring.io/spring-data/jpa/reference/>
- Spring. (s. f.-c). *Spring Initializr*. <https://start.spring.io/>
- springdoc-openapi. (s. f.). *springdoc-openapi* (Versión 3.1). <https://springdoc.org/>
- Testcontainers. (s. f.). *Testcontainers for Java*. <https://java.testcontainers.org/>

Anexo A. Capturas de las pruebas

Este anexo completa la evidencia del capítulo 3: reúne la captura de cada petición de la Tabla 2 que no aparece en el cuerpo, junto con el proyecto, la base de datos y las salidas completas de las suites automáticas. Las imágenes están a resolución completa y se pueden ampliar en el PDF.

A.1 Peticiones de la colección de Postman

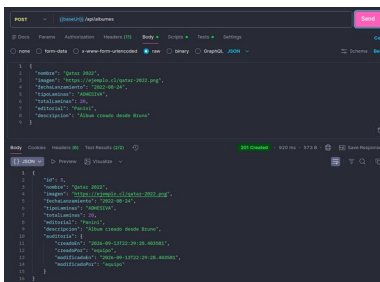


Figura 21: Crear álbum (201) (ev. 12).

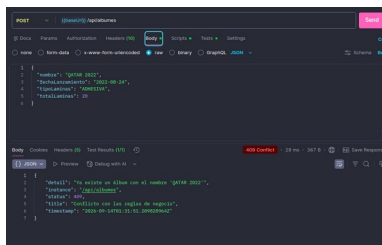


Figura 22: Nombre duplicado (409) (ev. 14).

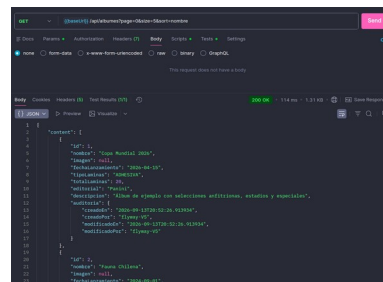


Figura 23: Listar álbumes paginado (200) (ev. 15).

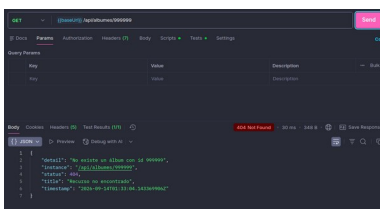


Figura 24: Álbum inexistente (404) (ev. 16).

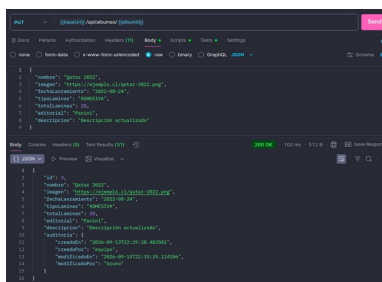


Figura 25: Actualizar álbum (200) (ev. 17).

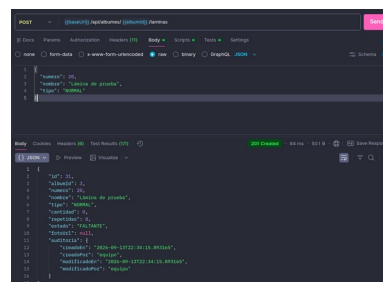


Figura 26: Crear lámina (201) (ev. 18).

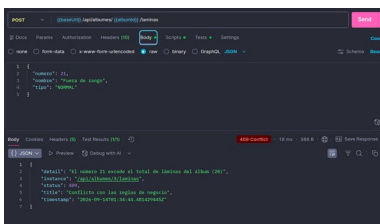


Figura 27: Número fuera de rango (409) (ev. 19).

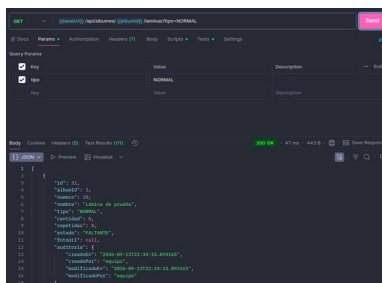


Figura 28: Listar láminas por tipo (200) (ev. 20).

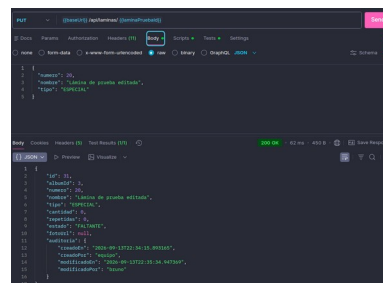


Figura 29: Actualizar lámina (200) (ev. 21).

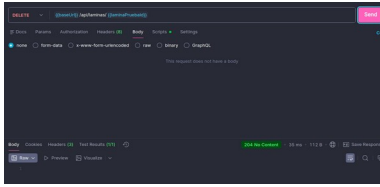


Figura 30: Eliminar lámina (204) (ev. 22).

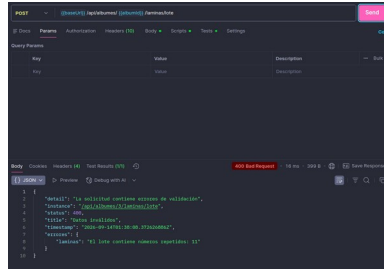


Figura 31: Lote con números repetidos (400) (ev. 25).

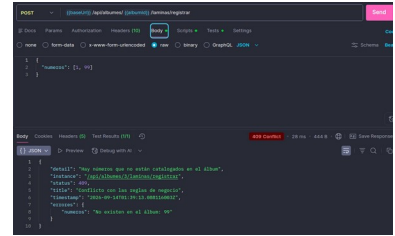


Figura 32: Registrar número no catalogado (409) (ev. 27).

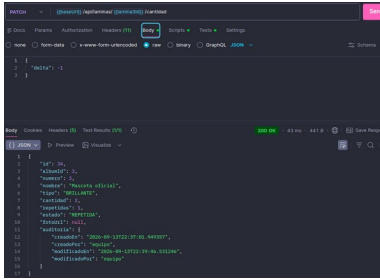


Figura 33: Restar una copia (200) (ev. 28).

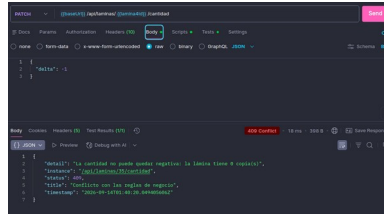


Figura 34: Cantidad negativa (409) (ev. 29).

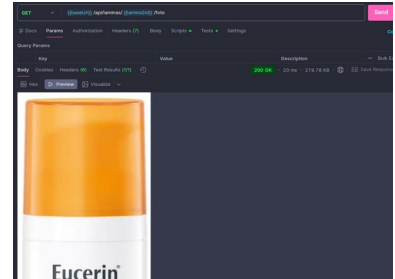


Figura 35: Ver foto (200) (ev. 34).

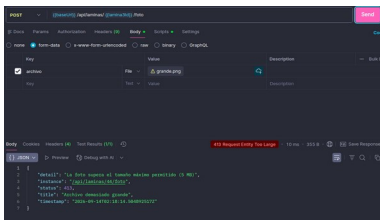


Figura 36: Archivo mayor a 5 MB (413) (ev. 36).

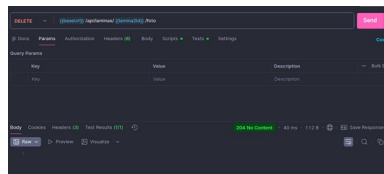


Figura 37: Quitar foto (204) (ev. 37).

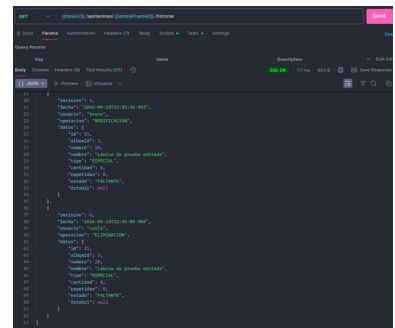


Figura 38: Historial de lámina eliminada (200) (ev. 39).

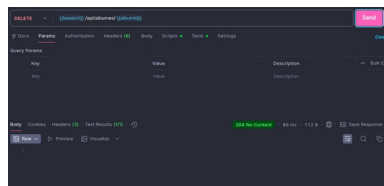


Figura 39: Eliminar álbum (204) (ev. 40).

A.2 Proyecto, base de datos y suites automáticas

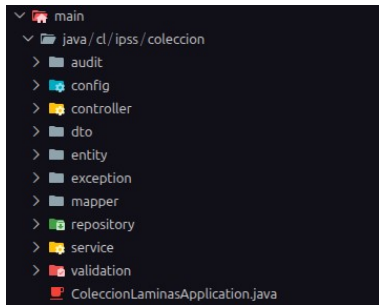


Figura 40: Estructura de paquetes del proyecto (ev. 04).

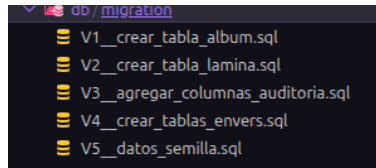


Figura 41: Scripts de migración en db/migration (ev. 07).

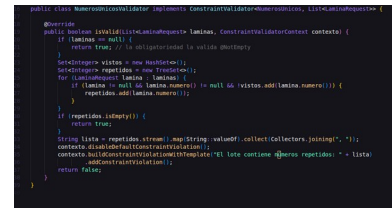


Figura 42: Validador personalizado NumerosUnicosValidator (ev. 43).

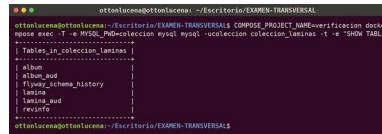


Figura 43: Tablas creadas por Flyway en MySQL (ev. 09).

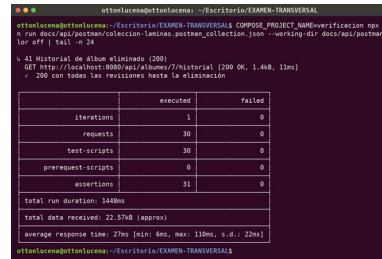


Figura 44: Final de la salida de Newman (colección Postman) (ev. 46).

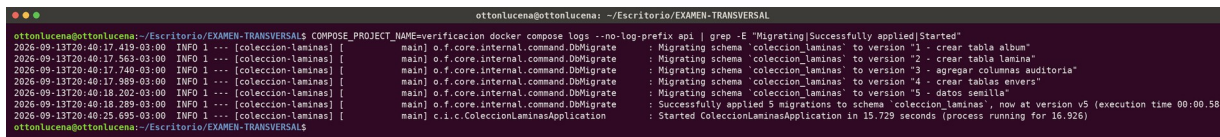


Figura 45: Arranque de la API: Flyway aplica V1 a V5 (ev. 06).

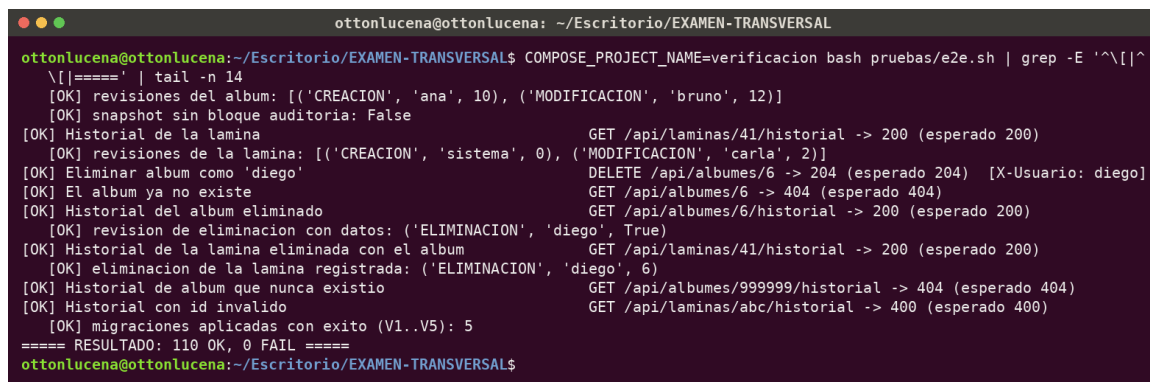


Figura 46: Final del script pruebas/e2e.sh (ev. 44).

```
ottonlucena@ottonlucena: ~/Escritorio/EXAMEN-TRANSVERSAL
ottonlucena@ottonlucena:~/Escritorio/EXAMEN-TRANSVERSAL$ cd backend && ./mvnw verify 2>&1 | grep -E "Running |Tests run|BUILD|Total time"
[INFO] Running cl.ipss.coleccion.service.impl.ColeccionServiceImplTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.650 s -- in cl.ipss.coleccion.service.impl.ColeccionServiceImplTest
[INFO] Running cl.ipss.coleccion.ColeccionLaminasApplicationTests
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 27.44 s -- in cl.ipss.coleccion.ColeccionLaminasApplicationTests
[INFO] Running cl.ipss.coleccion.controller.AlbumControllerTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.165 s -- in cl.ipss.coleccion.controller.AlbumControllerTest
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 36.561 s
ottonlucena@ottonlucena:~/Escritorio/EXAMEN-TRANSVERSAL$
```

Figura 47: Salida de ./mvnw verify (ev. 45).